

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for largescale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I, JASWANTH (192525345) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: I. JASWANTH

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1A:

Comparison of Sequential File Organization and Heap File Organization

Sequential file organization and heap file organization are two common methods of storing records in a file. They mainly differ in the way records are arranged, inserted, deleted, and retrieved.

1. Sequential File Organization

In **sequential file organization**, records are stored one after another in a specific order, usually according to a **key field** such as Roll Number, Employee ID, or Account Number.

- Records are maintained in sorted order.
- Searching can be efficient because records follow a known order.
- It is suitable for processing large numbers of records sequentially.
- Insertion can be difficult because the new record must be placed in the correct position.
- Deletion may require reorganization to maintain the order.
- It is commonly used for reports, payroll systems, and applications requiring ordered data.

2. Heap File Organization

In **heap file organization**, records are stored in **no particular order**. A new record is generally placed in any available space or appended to the end of the file.

- Records are not sorted.
- New records can be inserted quickly.
- Deletion is relatively simple.
- Searching for a particular record may require scanning the entire file.
- It is suitable for applications where insertions are frequent and ordered retrieval is not important.

Comparison

Factor	Sequential File Organization	Heap File Organization
Record Arrangement	Records are stored in a specific order.	Records are stored in any available order.
Insertion	Slow because the record must be inserted in the correct position.	Fast because the record can be added to available space.
Deletion	Relatively slow because the order may need to be maintained.	Relatively fast because the record can simply be removed or marked deleted.
Searching	Faster because records are ordered and searching techniques can be applied.	Slower because records may need to be searched one by one.

Sequential Retrieval	Very efficient.	Less efficient compared with an ordered sequential file.
Range Queries	Efficient because records are sorted.	Less efficient because records are scattered.
Updating Records	Can be difficult if the update changes the key or record position.	Relatively simple if the record fits in its existing space.
Space Management	May require additional space or reorganization after insertion/deletion.	Makes use of available free space more easily.
Best Suitable For	Applications requiring frequent searching and ordered processing.	Applications requiring frequent insertion and deletion.

Retrieval Efficiency

Sequential File:

Retrieval is generally faster, especially when records are accessed in order. For example, if employee records are sorted by Employee ID, records can be processed efficiently from the beginning to the end.

Heap File:

Retrieval is slower for a particular record because there is no ordering. The system may need to examine many records before finding the required record.

Insertion Efficiency

Sequential File:

Insertion is comparatively slow. If a new record belongs between two existing records, space may have to be created or the file may need to be reorganized.

Heap File:

Insertion is very fast. A new record can normally be placed in the first available free space or at the end of the file.

Deletion Efficiency

Sequential File:

Deletion is more complicated because the organization of the file should remain consistent. Deleted space may need to be managed or the file may require reorganization.

Heap File:

Deletion is easier. The record can be removed or marked as deleted, and its space can later be reused.

Conclusion

Sequential file organization provides better retrieval efficiency, particularly for sequential processing and range-based searches, but insertion and deletion are comparatively slower. Heap file organization provides faster insertion and deletion, but retrieval of a specific record is generally slower.

Therefore, sequential organization is preferred when fast and ordered retrieval is important, while heap organization is preferred when frequent insertions and deletions are more important than search performance.

2A:

Clustered Indexing vs Non-Clustered Indexing

Indexing is a database technique used to improve the speed of data retrieval. The two common types are **clustered indexing** and **non-clustered indexing**.

1. Clustered Indexing

A **clustered index** determines the physical order in which records are stored in a table. The actual data rows are stored according to the index key.

Example:

Suppose we have a Student table:

Student_ID	Name	Department
101	Arun	CSE
102	Ravi	ECE
103	Priya	CSE
104	Kiran	IT

If Student_ID is the clustered index, the records are physically organized according to Student_ID.

```
CREATE CLUSTERED INDEX idx_student_id  
ON Student (Student_ID);
```

Searching for Student_ID = 103 becomes efficient because the records are organized based on the indexed field.

2. Non-Clustered Indexing

A **non-clustered index** does not change the physical order of records in the table. Instead, it creates a separate index structure containing the indexed values and pointers/references to the actual records.

Example:

If we frequently search students based on their department, we can create a non-clustered index:

```
CREATE NONCLUSTERED INDEX idx_department  
ON Student(Department);
```

The database maintains a separate index for Department that points to the corresponding rows.

Difference Between Clustered and Non-Clustered Indexing

Feature	Clustered Index	Non-Clustered Index
---------	-----------------	---------------------

Data Storage	Determines the physical order of data rows.	Data rows remain in their original order.
Number per Table	Usually only one clustered index because data can have only one physical order.	Multiple non-clustered indexes can be created.
Speed	Very fast for range and ordered searches.	Fast for specific searches but may require an additional lookup.
Storage	Usually does not require a completely separate copy of the table data.	Requires additional storage for the index structure.
Insertion	Can be slower because records may need to maintain the physical order.	Usually faster than clustered indexing, although the index must also be updated.
Deletion	May require maintaining physical organization.	Index entries must be removed or updated.
Range Queries	Highly efficient.	Efficient, but may require additional data lookups.
Example	Index on Student_ID.	Index on Department or Name.
Best Used For	Primary keys and frequently used range queries.	Frequently searched non-key columns.

Suitable Database Example

Consider an **Employee** table:

```
CREATE TABLE Employee (
    Emp_ID INT,
    Emp_Name VARCHAR(50),
    Department VARCHAR(30),
    Salary INT
);
```

A clustered index can be created on Emp_ID: CREATE CLUSTERED INDEX idx_emp_id ON Employee(Emp_ID);

A non-clustered index can be created on Department: CREATE NONCLUSTERED INDEX idx_department ON Employee(Department); For a query such as:

```
SELECT * FROM Employee WHERE
Emp_ID = 105;
```

the **clustered index** can provide efficient access because the records are organized by Emp_ID.

For:

SELECT * FROM Employee WHERE Department = 'CSE'; the **non-clustered index** on Department helps locate the required employee records without changing the physical order of the table.

Conclusion

Clustered indexing stores table records in the order of the index key and is highly efficient for ordered and range-based retrieval. **Non-clustered indexing** maintains a separate index structure pointing to the actual records and is useful when multiple columns need to be searched efficiently.

In simple terms:

- **Clustered index** → **Data itself is arranged according to the index.**
- **Non-clustered index** → **Separate index points to the data.**

3A:

Primary Indexing vs Secondary Indexing

Indexing is a technique used in databases to improve the speed of searching and retrieving records. **Primary indexing** is created on the primary key or ordering key, while **secondary indexing** is created on a non-ordering field.

Comparison

Basis	Primary Indexing	Secondary Indexing
Definition	Index created on the field according to which the data file is physically ordered.	Index created on a field that is not used to physically order the data.
Storage Requirement	Requires less storage because it can often use a sparse index , with one index entry for a block.	Requires more storage because it generally needs a dense index , with an index entry for each record or search value.
Search Performance	Very fast because the data is ordered according to the index field.	Fast, but may require an additional pointer lookup to locate the actual record.
Index Size	Smaller.	Larger.
Data Ordering	Data records are sorted according to the primary index key.	Data records need not be sorted according to the secondary index key.
Number of Indexes	Usually one primary index for a particular ordering of the file.	Multiple secondary indexes can be created on different fields.

Range Search	Highly efficient because records are stored in sorted order.	Less efficient compared with primary indexing.
Example	Index on Student_ID when records are stored in Student_ID order.	Index on Student_Name or Department when records are stored in Student_ID order.

Example

Consider a Student table:

Student_ID	Name	Department
101	Arun	CSE
102	Ravi	ECE
103	Priya	CSE
104	Kiran	IT

If the file is physically sorted by Student_ID, an index on Student_ID is a **primary index**.

A secondary index can be created on Department to quickly find all CSE students, even though the records are not physically arranged by Department.

Storage Requirement

Primary indexing requires less storage because it can use a sparse index. For example, if a data block contains 100 records, the index may store only one entry representing that block.

Secondary indexing generally requires more storage because multiple records may have the same value, and the index needs additional pointers to locate all matching records.

Search Performance

Primary indexing provides **very efficient searching**, especially for ordered and range-based searches.

Secondary indexing also improves search performance significantly compared with scanning the entire file, but it may require an additional step to follow pointers from the index to the actual records.

Conclusion

Primary indexing requires **less storage and provides very efficient search performance** because the data is physically ordered according to the index key. **Secondary indexing** requires **more storage but provides flexible searching** on non-ordering fields.

In simple terms:

- **Primary Index** → **Less storage + very fast ordered search**
- **Secondary Index** → **More storage + flexible search on other fields**

4A:

Static Hashing vs Dynamic Hashing in Database Systems

Hashing is a database technique used to store and retrieve records quickly by applying a **hash function** to a search key. Based on how the number of storage buckets is handled, hashing can be classified as **static hashing** and **dynamic hashing**.

1. Static Hashing

In **static hashing**, the number of buckets is fixed when the hash file is created. A hash function maps each record to one of these fixed buckets.

Example:

Suppose a database has 10 buckets and the hash function is:

$h(\text{Student_ID}) = \text{Student_ID} \text{ MOD } 10$ For

$\text{Student_ID} = 125$:

$125 \text{ MOD } 10 = 5$

Therefore, the record is stored in **Bucket 5**.

As more records are inserted, some buckets may become full. This can result in **overflow buckets** and additional searching.

2. Dynamic Hashing

In **dynamic hashing**, the number of buckets can increase or decrease according to the amount of data stored. It is designed to handle databases where the number of records changes frequently.

When a bucket becomes full, the system can **split the bucket** and redistribute records. This reduces overflow and maintains efficient access.

Examples of dynamic hashing techniques include:

- **Extendible hashing**
- **Linear hashing**

Detailed Comparison

Feature	Static Hashing	Dynamic Hashing
Number of Buckets	Fixed	Can increase or decrease

File Growth	Not easily handled	Easily handles growth
Hash Function	Usually remains fixed	Can adapt as the file grows
Insertion	May become slower when buckets overflow	Generally efficient because buckets can split
Deletion	May leave unused space	Buckets can potentially be merged or reorganized
Overflow	More likely	Less likely
Storage Utilization	May become inefficient as data changes	Better utilization of storage
Search Performance	Fast when there is little overflow	Consistently fast as the database grows
Implementation	Simple	More complex
Maintenance	Easy to implement and maintain	Requires bucket splitting/merging
Best Suitable For	Databases with relatively stable data size	Databases with frequently changing data size

Advantages of Static Hashing

1. Simple to implement.
2. Easy to understand and maintain.
3. Provides fast access when the number of records is relatively stable.
4. Requires less management overhead.
5. Suitable for fixed-size databases.

Disadvantages of Static Hashing

1. Fixed number of buckets limits scalability.
2. Overflow can occur when the database grows.
3. Too many overflow buckets can reduce search performance.
4. Storage may be wasted when many buckets are partially empty.
5. Reorganizing the entire hash file may be necessary when the database grows significantly.

Advantages of Dynamic Hashing

1. Supports database growth efficiently.
2. Reduces overflow problems.
3. Provides better storage utilization.
4. Maintains good search performance as records increase.
5. Suitable for frequently changing databases.

Disadvantages of Dynamic Hashing

1. More complex to implement.
2. Bucket splitting and merging require additional processing.

3. Requires additional metadata to manage the changing structure.
4. Implementation and maintenance are more complicated than static hashing.

Example Application

Consider a **bank database** storing customer accounts.

- If the bank has a fixed number of accounts and the number changes very little, **static hashing** may be sufficient.
- If thousands of new accounts are added regularly, **dynamic hashing** is more suitable because the storage structure can grow along with the database.

Conclusion

Static hashing is simple and efficient for databases with a relatively stable number of records, but its fixed bucket structure can cause overflow as the database grows. **Dynamic hashing** is more flexible because it allows the number of buckets to change dynamically, providing better scalability and storage utilization.

In simple terms:

Static Hashing → Fixed buckets → Simple but less scalable

Dynamic Hashing → Flexible buckets → More complex but highly scalable

5A:

B-Tree Indexing vs B+ Tree Indexing for Large-Scale Database Applications

B-Tree and B+ Tree are balanced tree-based indexing techniques used in database systems to efficiently search, insert, and delete records. Both maintain a balanced structure, but they differ in how data is stored and accessed.

Comparison

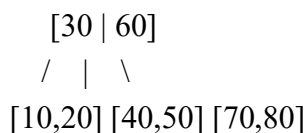
Feature	B-Tree	B+ Tree
Data Storage	Data records or pointers can be stored in both internal and leaf nodes.	Data records or pointers are stored only in leaf nodes.
Internal Nodes	Store keys and data pointers.	Store only keys and child pointers.
Leaf Nodes	May or may not be linked.	Leaf nodes are usually linked sequentially.
Search	Fast because data can be found at internal or leaf nodes.	Very efficient and predictable because searches reach leaf nodes.
Range Queries	Less efficient than B+ Tree.	Highly efficient due to linked leaf nodes.
Sequential Access	Comparatively slower.	Faster because leaf nodes are linked.
Fan-out	Lower because internal nodes store data pointers.	Higher because internal nodes store only keys and pointers.

Tree Height	May be slightly higher.	Usually smaller due to higher fanout.
Disk I/O	May require more disk accesses.	Generally requires fewer disk accesses.
Insertion	Requires node splitting when nodes become full.	Requires splitting and may propagate changes upward.
Deletion	Requires maintaining tree balance.	Requires maintaining balance and leaf-level structure.
Large Databases	Suitable, but less commonly preferred for database indexing.	Highly suitable and widely used for large-scale database systems.

B-Tree

A **B-Tree** is a balanced multiway search tree where both internal nodes and leaf nodes can contain keys and pointers to actual records.

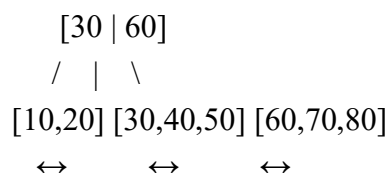
For example:



A search can sometimes find the required record at an internal node itself.

B+ Tree

In a **B+ Tree**, internal nodes contain only search keys and pointers, while the actual record pointers are stored at the leaf level.



The leaf nodes are linked, making sequential and range-based retrieval very efficient.

Performance in Large-Scale Databases

For large-scale databases, **B+ Trees generally provide better performance** because their internal nodes contain only keys and pointers. This allows more keys to fit into each disk page, increasing the **fan-out** and reducing the height of the tree.

For example, a query such as:

```

SELECT * FROM Employee
WHERE Salary BETWEEN 40000 AND 60000;

```

can be efficiently processed using a B+ Tree because the database can locate the starting leaf node and then follow the linked leaf nodes to retrieve the remaining records in the required range.

Advantages of B-Tree

- Supports fast direct searching.
- Balanced structure provides efficient operations.
- Suitable for general-purpose indexing.

- Searching can terminate at an internal node.
- Provides efficient insertion and deletion.

Advantages of B+ Tree

- Higher fan-out reduces tree height.
- Requires fewer disk I/O operations.
- Excellent for range queries.
- Linked leaf nodes provide fast sequential access.
- Well suited for very large databases.
- Provides predictable search performance.

Conclusion

Both **B-Tree** and **B+ Tree** provide efficient searching, insertion, and deletion with approximately $O(\log n)$ complexity. However, **B+ Tree indexing is generally preferred for large-scale database applications** because it has higher fan-out, lower tree height, better disk utilization, and highly efficient range and sequential queries.

In simple terms:

B-Tree → Data can exist in internal and leaf nodes.

B+ Tree → Data pointers are at leaf nodes + leaf nodes are linked → better for large databases.

6A:

Linear Hashing vs Extendible Hashing

Linear hashing and **extendible hashing** are dynamic hashing techniques used in database systems. Both allow the number of buckets to grow as data increases, but they manage buckets differently.

Basis	Linear Hashing	Extendible Hashing
Bucket Management	Buckets are split gradually in a predetermined linear order.	Buckets are split based on hash values and a directory.
Directory	Does not require a directory.	Uses a directory containing pointers to buckets.
Bucket Splitting	One bucket is split at a time when required.	A bucket is split when it overflows; directory may also double.
Growth	Grows gradually and smoothly.	Can grow rapidly when the directory doubles.
Scalability	Highly scalable for continuously growing files.	Highly scalable, especially for unpredictable growth.
Memory Requirement	Lower because no directory is maintained.	Higher because a directory must be maintained.

Overflow Handling	Uses gradual bucket splitting and may temporarily use overflow pages.	Bucket splitting reduces overflow and redistributes records.
Implementation	Relatively simple.	More complex due to directory management.
Search Performance	Good and generally stable.	Very good because the directory directly identifies the required bucket.
Best Suitable For	Databases with continuous and gradual growth.	Databases requiring fast access and flexible growth.

Linear Hashing

In **linear hashing**, buckets are split one by one in a fixed sequence. It does not require a directory.

For example:

Initial: B0 B1 B2

After split: B0 B1 B2 B3

↑

New bucket

The splitting process continues gradually as more records are inserted. This makes linear hashing useful for databases where data grows continuously.

Extendible Hashing

In **extendible hashing**, a **directory** is used to identify the appropriate bucket. The directory uses bits from the hash value.

When a bucket becomes full:

1. The bucket is split.
2. Records are redistributed.
3. If necessary, the directory size is doubled.
4. Directory pointers are updated.

Example:

Directory

00 —→ B0

01 —→ B1

10 —→ B2

11 —→ B3

Scalability Analysis

Linear hashing provides smooth scalability because only one bucket is generally split at a time. It avoids the need for a large directory, so memory usage is lower.

Extendible hashing can handle rapid database growth efficiently because buckets can split whenever they become full. However, directory doubling can increase memory requirements.

Conclusion

Linear hashing is simpler, uses less additional memory, and provides gradual growth.

Extendible hashing provides faster and more direct bucket access through its directory but requires additional memory and management.

In simple terms:

- **Linear Hashing → Gradual bucket splitting → Simple and memory-efficient**
- **Extendible Hashing → Directory-based bucket splitting → Fast and highly flexible**

7A:

Dense Indexing vs Sparse Indexing

Dense indexing and **sparse indexing** are two types of database indexing techniques. They differ mainly in the number of index entries stored, memory usage, and data access speed.

Comparison

Basis	Dense Indexing	Sparse Indexing
Index Entries	Contains an index entry for every record or search-key value.	Contains index entries only for some records, usually one entry per data block.
Memory Usage	Requires more memory because the index is larger.	Requires less memory because fewer index entries are stored.
Access Speed	Faster because the required record can be located directly through the index.	Slightly slower because the index identifies a block, after which the block must be searched.
Search Time	Generally very fast.	Slightly slower than dense indexing.
Index Size	Large.	Small.
Storage Requirement	High.	Low.

Data Ordering	Can be used with ordered or unordered data depending on the index type.	Usually used with files where records are stored in sorted order.
Maintenance	More index entries must be updated during insertions and deletions.	Fewer entries need to be maintained.
Suitable For	Applications where fast retrieval is the main requirement.	Large databases where saving memory is important.

Dense Indexing

In **dense indexing**, every record has a corresponding index entry.

Example:

Student ID Index

101 → Record 101

102 → Record 102

103 → Record 103

104 → Record 104

If we search for Student ID **103**, the index directly points to the required record. Therefore, access is very fast, but the index consumes more memory.

Sparse Indexing

In **sparse indexing**, only selected records are indexed, commonly one entry for each data block.

Example:

Index

101 → Block 1

201 → Block 2

301 → Block 3

If we search for Student ID **250**, the index identifies **Block 2**, and then the block is searched to find record 250.

Therefore, sparse indexing requires less memory but may take slightly more time for retrieval.

Memory and Access Speed Dense Indexing:

- Uses **more memory**.
- Provides **faster access**.
- Suitable when quick record retrieval is important.

Sparse Indexing:

- Uses **less memory**.
- Provides **slightly slower access**.
- Suitable for large files where reducing index storage is important.

Conclusion

Dense indexing provides **higher access speed at the cost of greater memory usage**, while sparse indexing provides **lower memory usage with slightly slower access**.

In simple terms:

Dense Index → More memory + Faster access

Sparse Index → Less memory + Slightly slower access

8A:

Single-Level Indexing vs Multi-Level Indexing

Indexing is a technique used in database systems to improve the speed of data retrieval.

Single-level indexing uses one index level, while **multi-level indexing** uses multiple levels of indexes to reduce the search time for large databases.

Comparison

Basis	Single-Level Indexing	Multi-Level Indexing
Structure	Contains only one index level.	Contains two or more levels of indexes.
Search Process	Search is performed directly in the single index.	Higher-level index is searched first, followed by lower-level indexes.
Access Speed	Fast for small databases.	Faster and more efficient for very large databases.
Memory Usage	Requires more memory when the index becomes very large.	Reduces the amount of index that must be searched at each level.

Index Size	Can become large as the number of records increases.	Each higher-level index is smaller than the level below it.
Disk I/O	May require more disk accesses for a large index.	Reduces disk I/O by narrowing the search at each level.
Implementation	Simple and easy to implement.	More complex to implement and maintain.
Scalability	Suitable for small or medium-sized files.	Highly suitable for large-scale databases.
Maintenance	Easier to maintain.	Requires maintenance at multiple index levels.
Example	One index on Student_ID.	An index on the index of Student_ID.

Single-Level Indexing

In **single-level indexing**, one index file is created for the data file.

Example:

Single-Level Index



Data File

Suppose a student database contains thousands of records. The index may contain:

101 → Record 1

201 → Record 2

301 → Record 3

401 → Record 4

When searching for a record, the database searches this single index and then accesses the required data record.

For small databases, this method is simple and effective. However, when the index itself becomes very large, searching the index can require more time and disk I/O.

Multi-Level Indexing

In **multi-level indexing**, an index is created on the first-level index itself.

Second-Level Index



First-Level Index



Data File

For example:

Level 2 Index



Level 1 Index



Data Blocks

The database first searches the **second-level index** to identify the appropriate portion of the first-level index. It then searches the first-level index to locate the required data block.

This reduces the amount of index information that must be searched at one time.

Retrieval Performance

Consider a large database containing **1 million records**.

With **single-level indexing**, the database may have to search a large index before locating the required record.

With **multi-level indexing**, the search is divided into smaller steps:

1. Search the top-level index.
2. Identify the required lower-level index.
3. Search the lower-level index.
4. Locate the required data block.
5. Retrieve the record.

Therefore, multi-level indexing is more efficient for very large databases.

Advantages of Single-Level Indexing

- Simple structure.
- Easy to implement.
- Easy to maintain.
- Suitable for small and medium-sized databases.
- Requires fewer levels of management.

Advantages of Multi-Level Indexing •

Faster retrieval for large databases.

- Reduces disk I/O.
- Reduces the amount of index searched at each step.
- Suitable for very large data files.
- Provides better scalability.

Conclusion

Single-level indexing is simple and effective for small databases, but its performance can decrease when the index becomes very large. **Multi-level indexing** divides the index into multiple levels, reducing search time and disk accesses. Therefore, **multi-level indexing is more suitable for large-scale database retrieval operations.**

In simple terms:

Single-Level Indexing → One index → Simple but less efficient for huge databases

Multi-Level Indexing → Index of indexes → Faster and more scalable for large databases

9A:

Comparison of Single-Level and Multi-Level Indexing

Indexing is a database technique used to speed up the retrieval of records. Single-level indexing uses one index, whereas multi-level indexing uses multiple levels of indexes.

Basis	Single-Level Indexing	Multi-Level Indexing
Structure	Contains one index level.	Contains multiple index levels.
Search Method	Searches the single index directly.	Searches the top-level index, then lowerlevel indexes.
Access Speed	Fast for small databases.	Faster for large databases.
Memory Usage	The index can become large.	Higher-level indexes are smaller, reducing the search space.
Disk I/O	May require more I/O when the index is large.	Requires fewer disk accesses.

Implementation	Simple and easy to implement.	More complex to implement.
Maintenance	Easy to maintain.	More difficult because multiple levels must be updated.
Scalability	Suitable for small and medium-sized databases.	Suitable for large-scale databases.
Search Efficiency	Decreases as the index grows.	Remains efficient even for very large files.
Example	One index points directly to data blocks.	A top-level index points to lower-level indexes, which point to data blocks.

Single-Level Indexing

In single-level indexing, a single index is maintained for the data file.

Index



Data File

For example, an index on Student_ID may contain:

101 → Block 1

201 → Block 2

301 → Block 3

When the user searches for a student, the database searches this index and then accesses the appropriate data block.

Advantages:

- Simple structure.
- Easy to implement.
- Easy to maintain.
- Suitable for smaller databases.

Disadvantage: As the number of records increases, the index becomes large, which can increase search time and disk I/O.

Multi-Level Indexing

In multi-level indexing, an index is created on another index.

Second-Level Index



First-Level Index



Data File

The database first searches the higher-level index to locate the required section of the lower-level index. It then searches the lower-level index to find the required data block.

Advantages:

- Faster searching for large databases.

- Reduces disk I/O.
- Handles very large indexes efficiently.
- Provides better scalability.

Disadvantage: It is more complex and requires additional maintenance when records are inserted or deleted.

Example

Suppose a database contains **1 million student records**.

With **single-level indexing**, the database may need to search a very large index.

With **multi-level indexing**, the search is divided into smaller steps: **Top-level**

index → **Lower-level index** → **Data block** → **Required record** Thus,

multi-level indexing reduces the amount of searching required.

Conclusion

Single-level indexing is **simple and suitable for small or medium-sized databases**, but its performance may decrease as the index grows. Multi-level indexing provides **faster and more efficient retrieval for large databases** by dividing the index into multiple levels and reducing disk access.

In short:

Single-Level Indexing → Simple + Easy to maintain

Multi-Level Indexing → Faster + More scalable for large databases

10A:

Performance of Hashing and Indexing Techniques for Exact-Match and Range Queries

Hashing and indexing are important techniques used to improve database query performance. However, they are suitable for different types of queries. **Hashing is highly efficient for exact-match queries**, while **indexing, especially B+ Tree indexing, is better for range queries**.

Comparison

Query Type	Hashing	Indexing (B+ Tree)
Exact-Match Query	Very fast, usually O(1) average access time.	Fast, generally O(log n) .
Range Query	Poor performance because hash values do not preserve order.	Excellent because keys are stored in sorted order.

Example	WHERE Student_ID = 105	WHERE Student_ID = 105
Range Example	WHERE Salary BETWEEN 30000 AND 50000 is inefficient.	WHERE Salary BETWEEN 30000 AND 50000 is efficient.
Ordering	Does not maintain key order.	Maintains keys in sorted order.
Search Performance	Best for exact matching.	Good for both exact and range searches.
Storage	Requires hash buckets and may require overflow handling.	Requires additional index pages and pointers.
Scalability	Dynamic hashing handles growing data effectively.	B+ Trees scale well for large databases.
Best Use	Exact-match searches.	Range, ordered, and exact-match searches.

1. Hashing for Exact-Match Queries

Hashing uses a **hash function** to determine the location of a record.

For example: $h(\text{Student_ID}) =$

$\text{Student_ID} \text{ MOD } 10$ For:

$\text{Student_ID} = 105$

The hash function directly identifies the bucket containing the record.

A query such as:

SELECT * FROM Student WHERE Student_ID = 105; can be processed very quickly because the database directly calculates the required bucket. **Performance:** Average **O(1)** lookup, assuming a good hash function and limited collisions.

2. Hashing for Range Queries

Hashing is not suitable for range queries because it does not preserve the ordering of keys.

For example:

SELECT * FROM Employee

WHERE Salary BETWEEN 30000 AND 50000;

Employees with salaries 30,000, 40,000, and 50,000 may be placed in completely different hash buckets.

Therefore, the database may need to examine many or all buckets.

Performance: Approximately **O(n)** in the worst case for range retrieval.

3. Indexing for Exact-Match Queries

A **B+ Tree index** can also provide efficient exact-match searching.

For example:

SELECT * FROM Student

WHERE Student_ID = 105;

The database follows the B+ Tree from the root to the appropriate leaf node.

Performance: Approximately **O(log n)**.

Although hashing can be faster for exact matches, B+ Tree indexing still provides excellent performance.

4. Indexing for Range Queries

B+ Tree indexing is particularly effective for range queries because the leaf nodes contain keys in sorted order and are generally linked.

For example:

```
SELECT * FROM Employee
```

```
WHERE Salary BETWEEN 30000 AND 50000;
```

The database finds the first matching value and then scans the consecutive leaf entries until the end of the required range.

Therefore, B+ Tree indexes provide much better range-query performance than hashing.

Performance Analysis

Technique	Exact Match	Range Query
Static Hashing	Excellent	Poor
Dynamic Hashing	Excellent	Poor
B+ Tree Index	Very Good	Excellent
B-Tree Index	Very Good	Good
Sequential Index	Good	Very Good

Practical Example

Consider an **Employee** database containing millions of records.

Exact-match query:

```
SELECT * FROM Employee
```

```
WHERE Emp_ID = 50025;
```

A hash index is highly suitable because the employee record can be located directly through the hash value.

Range query:

```
SELECT * FROM Employee
```

```
WHERE Emp_ID BETWEEN 50000 AND 50100;
```

A B+ Tree index is more suitable because the employee IDs are ordered, allowing the database to efficiently retrieve all records within the specified range.

Conclusion

Hashing and indexing have different performance advantages:

- **Hashing → Best for exact-match queries**
- **B+ Tree indexing → Best for range queries**
- **Hashing generally provides $O(1)$ average exact-match access.**
- **B+ Tree provides $O(\log n)$ search and efficient range retrieval.**
- **For databases requiring both exact-match and range queries, B+ Tree indexing is generally more flexible.**

Overall: Hashing provides superior performance for direct exact searches, while indexing provides better and more consistent performance for range-based and ordered queries.